

Advanced Programming

Introduction to *JavaFX*

Álvaro Santos@DEIS-ISEC

Graphics programming in Java

- The creation of graphical applications in *Java* has been considered since its beginning
 - <https://www.oracle.com/java/technologies/painting.html>
- Initially the development of graphical environments was based on *AWT – Abstract Windowing Toolkit*
 - Despite this method still being used, its components are considered "*heavyweight* " because they are supported by native operating system code

Graphics programming in Java

- Later, its use evolved to a more modern framework called *Swing*
 - Components are considered "*lightweight*" because they are all developed in *Java*
 - Most of the components were developed on top of existing *AWT* components
 - The use of this API does not replace *AWT*, both can be used although it should be avoided

JavaFX

- *JavaFX* is a Java library for developing graphical applications (*Rich Internet Applications*), presented as an evolution and an alternative to *Swing*
 - The first version was released in October 2008
 - Framework is fully developed in *Java*
- Initially belonged to the *JDK* distribution, but started to be distributed separately from the JDK 11
 - *Oracle* shifted its development to *OpenJDK*, more specifically to a project called *OpenJFX*

JavaFX

- Main features:
 - Written in *Java*
 - FXML
 - *Scene Builder*
 - *Swing* Compatibility
 - Built-in Controls
 - CSS-like settings support
 - API's and set of classes to support 2D, 3D graphics and advanced content
 - Takes advantage of graphics acceleration mechanisms available on the device where the application runs

OpenJFX

- The most up-to-date documentation can be obtained from the website
 - <https://openjfx.io/>
- Integrating *JavaFX* into a *Java* project can be done by downloading the library and associating it with the project
 - The library consists of several *jar* files

Installation

- Download from *website* : <http://openjfx.io>
 - *Direct link* : <https://gluonhq.com/products/javafx/>
- For new projects, *download* an *LTS version* (21.0.6) or the latest version (24)
 - Download...
 - *SDK* for the desired operating system
 - Documentation (*JavaDoc*)
- Unzip to a directory, preferably near the *JDK* in use (the documentation can be unzipped to a `doc` directory within the created base directory)
 - *Windows* : `C:\Program Files\Java`
 - *MacOS* : `/Library/Java/JavaVirtualMachines`

JavaFX IntelliJ configuration

- Go to option *File* → *Project Structure*
 - *Global Libraries*
 - "+" → *New Java Library ...*
 - Enter the full path to the *JavaFX*
- Add the documentation *URLs* to the library
 - *URL* examples for local files:
 - `file:///C:\Program Files\Java\javafx-sdk-21.0.6\doc`
 - `file:///C:\Program Files\Java\javafx-sdk-21.0.6\doc\javafx.base`
 - `file:///C:\Program Files\Java\javafx-sdk-21.0.6\doc\javafx.controls`
 - `file:///C:\Program Files\Java\javafx-sdk-21.0.6\doc\javafx.fxml`
 - `file:///C:\Program Files\Java\javafx-sdk-21.0.6\doc\javafx.graphics`
 - `file:///C:\Program Files\Java\javafx-sdk-21.0.6\doc\javafx.media`
- These steps are performed only once

JavaFX project creation

- Create a *Java* project as done for previous *Java* projects
- In projects where *JavaFX* is to be used
 - *File* → *Project Structure*
 - *Global Libraries*
 - Right-click on the library and add it to the module (project created)

JavaFX project configuration

- In the run configurations
 - Add the following settings to the *VM Options*
 - `--module-path /path_to_javafx_sdk/lib`
 - `--add-modules javafx.controls`
 - if *xml* is used for the interface definition:
 - `--add-modules javafx.controls,javafx.fxml`
 - to include all the *JavaFX* modules
 - `--add-modules ALL-MODULE-PATH`

JavaFX

- A *JavaFX* application is encapsulated through a `javafx.application.Application` object
- Developing a *JavaFX* application usually starts by creating a new class that extends the `Application` class
 - The abstract method `void start(Stage)` must be defined
- The `Application` object is created by the system when the static method `Application.launch(...)` is executed
 - The `launch` method can only be called once in the context of a *JavaFX* application, i.e., there should only be one `Application` object

Application

- Example for when the class that derives from `Application` is the same one where the `main` function is defined

```
public class Main extends Application {  
  
    public static void main(String [] args) {  
        launch(args);  
    }  
  
    @Override  
    public void start(Stage stage) throws Exception {  
        // TODO  
    }  
}
```


Application

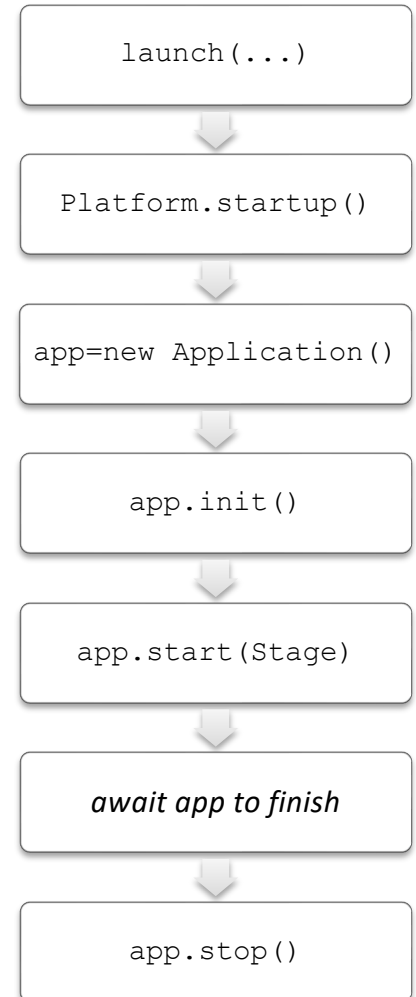
- Example for when the class that derives from `Application` is different from the class where the `main` function is defined

```
public class Main {  
    public static void main(String [] args) {  
        Application.launch(MainJFX.class, args);  
    }  
}
```

```
public class MainJFX extends Application {  
    @Override  
    public void start(Stage stage) throws Exception {  
        // TODO  
    }  
}
```

Lifecycle of an application

- When the `launch` method is called, the following actions are performed in order:
 - The *JavaFX runtime* is checked to ensure it has been properly started
 - If it has not yet been started, the `Platform.startup(...)` method is executed
 - An instance of the specified `Application` object is created
 - The `init()` method of the `Application` is called
 - This method can be overridden to execute initialization tasks and provision resources
 - The `start(Stage)` method of the `Application` is called
 - This method creates the entire graphical environment and configures behaviors
 - The application waits for termination. This occurs when...
 - The last window is closed, or ...
 - The `Platform.exit()` method is executed
 - The `stop()` method of the `Application` is called
 - This method releases allocated resources and performs other application termination tasks

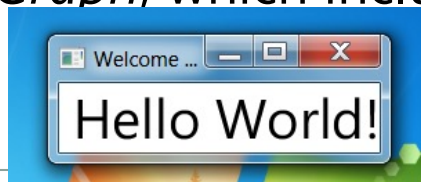


Stage, Scene and Scene Graph

- The graphical user interface of a JavaFX application must be created in the `start()` method of the `Application` object
- The objects that constitute the interface are organized through a tree of objects, which is made up of:
 - Stage
 - Scene
 - *Scene Graph*, which includes a *root node* and other *nodes*

Stage, Scene and Scene Graph

- The graphical user interface of a JavaFX application must be created in the `start()` method of the `Application` object
- The objects that constitute the interface are organized through a tree of objects, which is made up of:
 - `Stage`
 - Corresponds to the window that supports the application, adapted to the operating system
 - `Stages` can be created if several windows are needed
 - A previously created `Stage` object is received as a parameter in the `start` method
 - A `Scene` object must be associated with a `Stage` object
 - `Scene`
 - *Scene Graph*, which includes a *root node* and other *nodes*



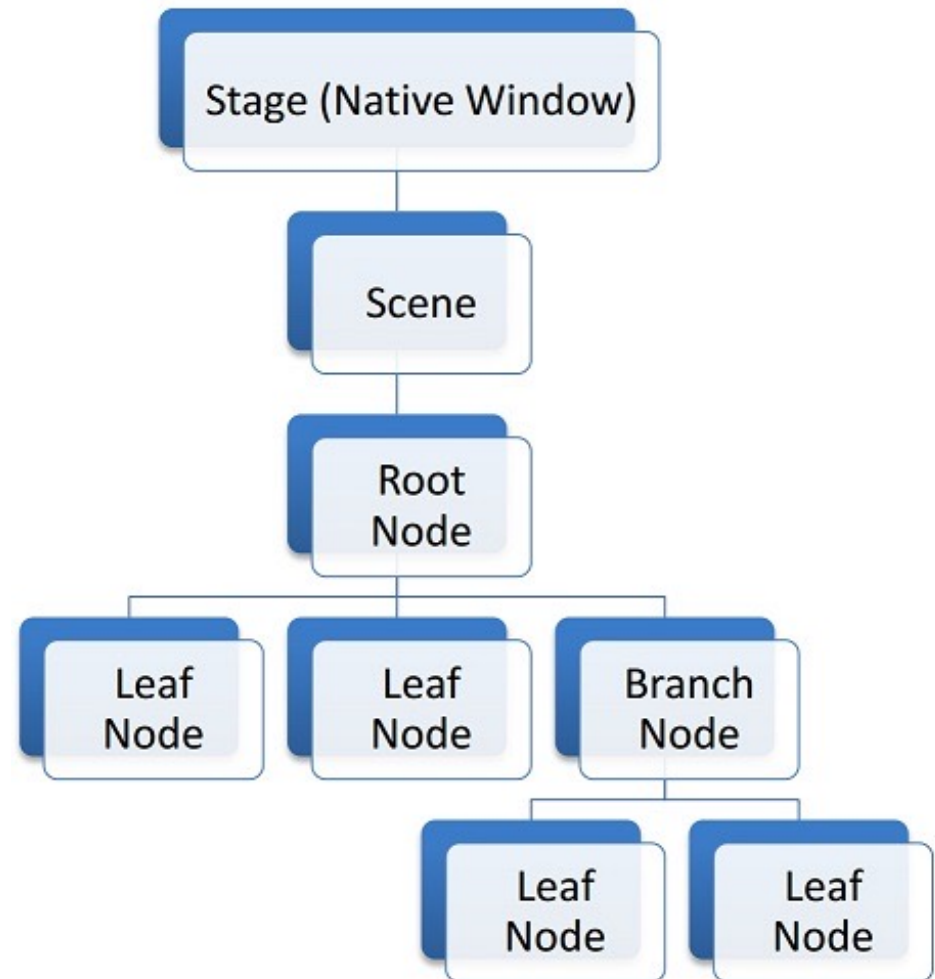
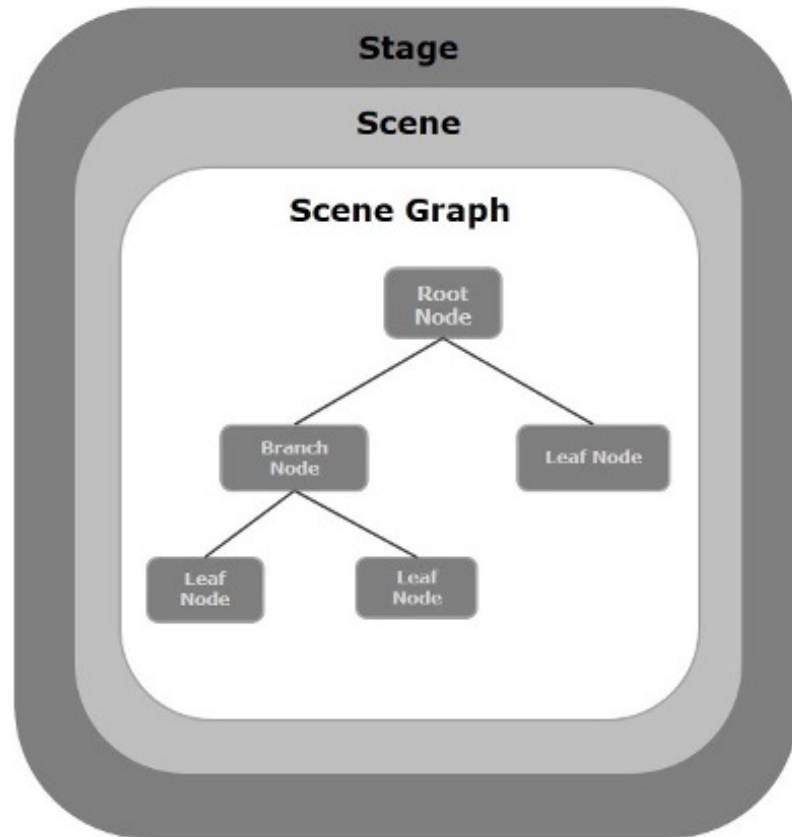
Stage, Scene and Scene Graph

- The graphical user interface of a JavaFX application must be created in the `start()` method of the `Application` object
- The objects that constitute the interface are organized through a tree of objects, which is made up of:
 - Stage
 - Scene
 - Represents the components that make up the *Scene graph*
 - A *width* and a *height* can be defined, thus defining the size of the support window
 - *Scene Graph*, which includes a *root node* and other *nodes*

Stage, Scene and Scene Graph

- The graphical user interface of a JavaFX application must be created in the `start()` method of the `Application` object
- The objects that constitute the interface are organized through a tree of objects, which is made up of:
 - `Stage`
 - `Scene`
 - *Scene Graph*, which includes a *root node* and other *nodes*
 - All components that make up the interface (buttons, text boxes, elements for organizing other elements, ...) derive directly or indirectly from the `Node` class
 - Some `Node` elements (*layout* elements – *pane*) can include other `Node` elements, thus creating a hierarchy of components
 - The first of these – the base of the hierarchy – is called *the Root Node*
 - This is the element that must be indicated when creating the `Scene` object

Stage, Scene and Scene Graph



https://www.tutorialspoint.com/javafx/javafx_application.htm

<https://fxdocs.github.io/docs/html5/>

JavaFX hierarchy

- `javafx.stage.Window`
 - `PopupWindow`
 - `Popup`
 - `PopupControl`
 - `ContextMenu`, `Tooltip`
 - `Stage`
- `javafx.scene.Scene`
- `javafx.scene.Node`
 - `Parent`
 - `Group`
 - `Region`
 - *Next slide...*
 - `WebView`
 - `Shape`
 - `Arc`, `Circle`, `CubicCurve`, `Ellipse`, `Line`, `Path`, `Polygon`, `Polyline`, `QuadCurve`, `Rectangle`, `SVGPath`, `Text`
 - `Shape3D`
 - `Box`, `Cylinder`, `MeshView`, `Sphere`
 - `Canvas`
 - `ImageView`
 - `Camera`, `LightBase`, `MediaView`, `SubScene`, `SwingNode`

JavaFX hierarchy

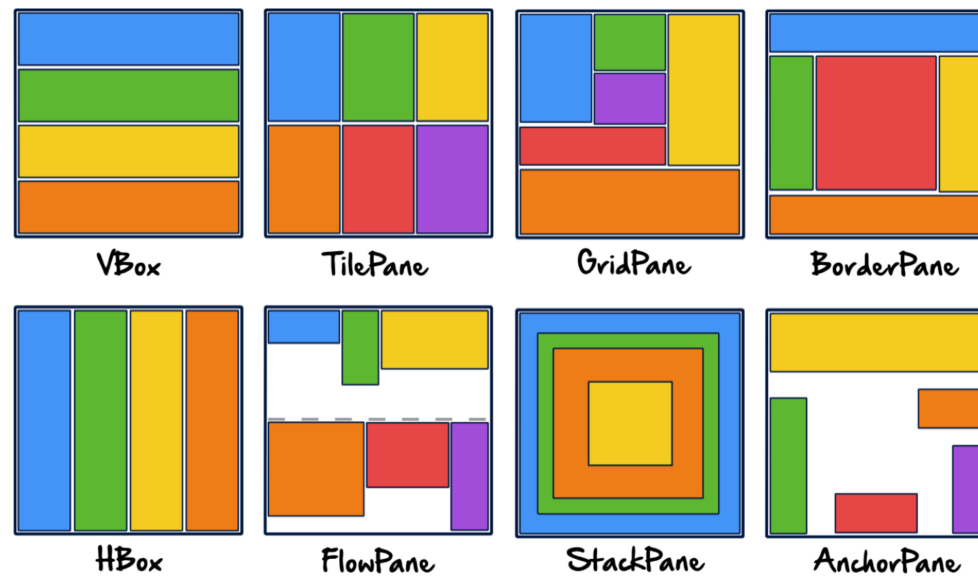
- Region
 - Control
 - TextInputControl
 - TextArea, TextField
 - ComboBoxBase
 - ColorPicker, ComboBox, DatePicker
 - Labeled
 - ButtonBase
 - Button
 - MenuButton
 - SplitMenuButton
 - ToggleButton
 - RadioButton
 - CheckBox
 - Hyperlink
 - Cell, Label, TitledPane
 - Accordion, ButtonBar, ChoiceBox, HTML editor, ListView, MenuBar, Pagination, ProgressIndicator, ScrollBar, ScrollPane, Separator, Slider, Spinner, SplitPane, TableView, TabPane, ToolBar, TreeTableView, TreeView
 - Pane
 - AnchorPane, BorderPane, DialogPane, FlowPane, GridPane, HBox, PopupControl.CSSBridge, StackPane, TextFlow, TilePane, VBox
 - Axis, Chart, TableColumnHeader, VirtualFlow

JavaFX hierarchy

- There are other *sub-hierarchies* with many other objects that make it possible to provide the usual functionalities and modes of interaction in current modern interfaces

Pane objects (*layout*)

- AnchorPane
- BorderPane
- FlowPane
- GridPane
- HBox
- StackPane
- TilePane
- VBox



layout formatting

- Depending on the type of layout object used, different formatting parameters and ways to add the components it generates (called children) are available
 - `getChildren().add, getChildren().addAll`
 - `setPadding`
 - `setAlignment`
 - `setSpacing`
 - `setTopAnchor, setBottomAnchor, setLeftAnchor, setRightAnchor`
 - `setTop, setBottom, setLeft, setRight, setCenter`
 - `setMargin`
 - ...

Formatting of Nodes

- Colors and other settings for each component can be configured using specific methods
 - `setBorder`
 - `setMaxSize`, `setMinWidth`, `setMaxHeight`, `setPrefHeight`, ...
 - `setText`, `setTextAlignment`, `setTextFill`
 - `setStyle("CSS style string")`
`obj.setStyle("-fx-background-color: #ffffd0;");`
 - ...

Example

```
public class MainJFX extends Application {  
    public static void main(String[] args) {  
        launch(args);  
    }  
    @Override  
    public void start(Stage stage) throws Exception {  
        BorderPane root = new BorderPane();  
        root.setStyle("-fx-background-color: #ffffe0;");  
        Label label = new Label("Advanced Programming");  
        label.setTextFill(Color.INDIGO);  
        label.setFont(new Font("Times New Roman", 24));  
        root.setCenter(label);  
        Scene scene = new Scene(root, 400, 300);  
        stage.setTitle("DEIS-ISEC");  
        stage.setResizable(false);  
        stage.setScene(scene);  
        stage.show();  
    }  
}
```



Event-driven programming

- In the context of *JavaFX*, as well as other graphical programming environments, ...
 - Application programming no longer follows a linear and well-defined sequence of instructions
 - Instead, it is based on asynchronous events that can occur on various components of the interface, typically as a result of user interaction (e.g., clicking a button)

Event-driven programming

- Event-driven programming consists of defining in advance what should be executed when an event occurs
 - Event-related objects provide suitable methods for defining what to do when events happen
 - `setOn*`, `addEventHandler`, `addEventFilter`
 - For example
 - If there is a button on the interface, the program does not wait in an "infinite" loop for the button to be pressed (this situation worsens if there are more buttons waiting to be pressed at the same time)
 - The code that should be executed when any of the buttons are pressed is indicated in advance
- Each event is encapsulated through a suitable object
 - For example, the event related to *clicking* on a button is represented through an `ActionEvent` object

ActionEvent

- As mentioned, when a `Button` is clicked, an event is generated represented by an `ActionEvent` object
- Event processing is performed through `EventHandler<T>` interface instances
 - The method `void handle(T event)` of the `EventHandler<T>` must be redefined, in the context of which the intended processing should be done
- There are several ways to create an `EventHandler<ActionEvent>` object, such as:
 - Create an instance of a class that implements the `EventHandler<ActionEvent>` interface
 - Create an *inline* object (*anonymous class*) that implements the `EventHandler<ActionEvent>` interface
 - *Lambda expression*

ActionEvent

- Example of a class that handles an ActionEvent

```
class MyHandler implements  
EventHandler<ActionEvent> {  
    @Override  
    public void handle(ActionEvent actionEvent) {  
        //TODO  
    }  
}
```

... which can be associated with a *button* as follows:

```
Button button = new Button("Go");  
button.setAction(new MyHandler());
```

ActionEvent

- Examples with *Lambda expressions*

- setOnAction

```
Button button = new Button("Increment");  
button.setOnAction(actionEvent -> {  
    //TODO  
});
```

- addEventFilter

```
Button button = new Button("Decrement");  
button.addEventFilter(  
    ActionEvent.ACTION,  
    actionEvent -> {  
        //TODO  
    }  
);
```

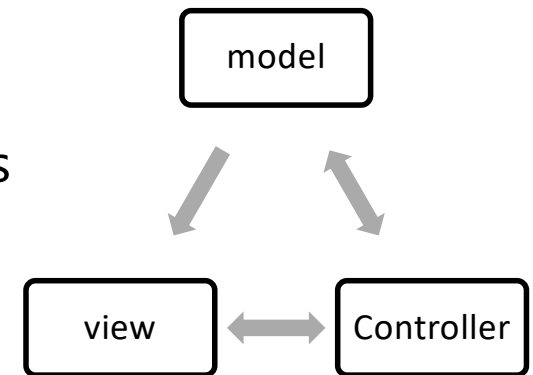
- With the addEventFilter/addEventHandler methods, different types of events can be handled and multiple processes can be configured for the same event

Project organization

- In order to maintain independence between the data model and the forms of visualization and/or interaction with the user, it is necessary to organize the entities that constitute the programs
- This separation/organization favors other aspects of development, for example:
 - Division of program complexity
 - Evolution of the different parts of the application independently
 - Allocation of different tasks to development teams or members
 - Code reuse
 - Application testing
 - Easy support
 - ...

Model-View-Controller (MVC)

- In the MVC pattern responsibilities are divided between:
 - Model
 - Set of classes that manages data, business logic/rules and respective algorithms
 - View
 - Classes and components that display relevant information at all times
 - Presents the necessary elements for the user to interact with the application
 - Controller
 - Performs the appropriate model tasks based on the user commands
 - Ensures that views are updated to reflect the most current information



- Sometimes it is difficult to distinguish between the "controller" and the "views"
 - This situation occurs since the programming of the information and configuration of the graphical aspect are carried out on the same objects in which the *listeners/handlers* of the events are configured

- In the context of the Advanced Programming subject the following *JavaFX* projects organization will be adopted:
 - *Model*
 - Set of classes that allow managing data, business logic/rules and respective algorithms
 - *View-Controller*
 - Class or classes that implement the various screens of the application's graphical interface

- (Additionally) To support the *JavaFX* application, the following *main classes* are defined
 - *Main*
 - Include the Java `main` method, which will call the JavaFX `Application.launch` method
 - *MainJFX*
 - Extends the JavaFX `Application` class
 - Responsible for configuring the `Stage` objects
 - Responsible for creating the first *View-Controller* object

MVC@PA – Model

- *Model*
 - Set of classes that allow managing data, business logic/rules and respective algorithms
 - These classes must not contain any code that interacts with the user ("neither *in* nor *out*")
 - Preferably, a *Facade* or similar class is provided, which allows all the internal specifics of the model to be hidden

MVC@PA – View-Controller

- *View-Controller*
 - Class or classes that implement the various screens of the application's graphical interface
 - Usually extends the `Pane` object or one of its derivatives, to make it possible to include the various appropriate graphical elements

MVC@PA – View-Controller

- *View-Controller*
 - In the constructor, aside from initializing the properties (based on parameters), the following methods should be called:
 - `createViews`
 - Responsible for creating views, configuring view properties, and creating the overall *layout of the interface*
 - `registerHandlers`
 - Association of *listeners/handlers* to the relevant events of the different elements of the created view
 - Which will act on the model to invoke the methods that will trigger the model's behaviors/methods
 - `update`
 - Method responsible for updating view information based on data obtained from the model (only calls to *get* methods of the model are expected)
 - For its proper functioning it is necessary that the *View-Controller* have a reference to the model (eventually the *Facade* or *Proxy* class that represents the model)

Project example

```
public class Main {  
    public static void main(String[] args) {  
        Application.launch(MainJFX.class, args);  
    }  
}
```

```
public class MainJFX extends Application {  
    ModelData data;  
  
    public MainJFX() { data = new ModelData(); } // It can also be created in 'init'  
  
    @Override  
    public void start(Stage stage) throws Exception {  
        RootPane root = new RootPane(data);  
        Scene scene = new Scene(root, 600, 400);  
        stage.setScene(scene);  
        stage.setTitle("TeoIntroJFX");  
        stage.show();  
    }  
}
```

Project example

```
public class RootPane extends VBox { //View-Controller
    ModelData data;
    // variables, including reference to views

    public RootPane(ModelData data) {
        this.data = data;

        createViews();
        registerHandlers();
        update();
    }

    private void createViews() { /* create and configure views */ }

    private void registerHandlers() { /* handlers/listeners */ }

    private void update() { /* update views */ }
}
```

Model evolution

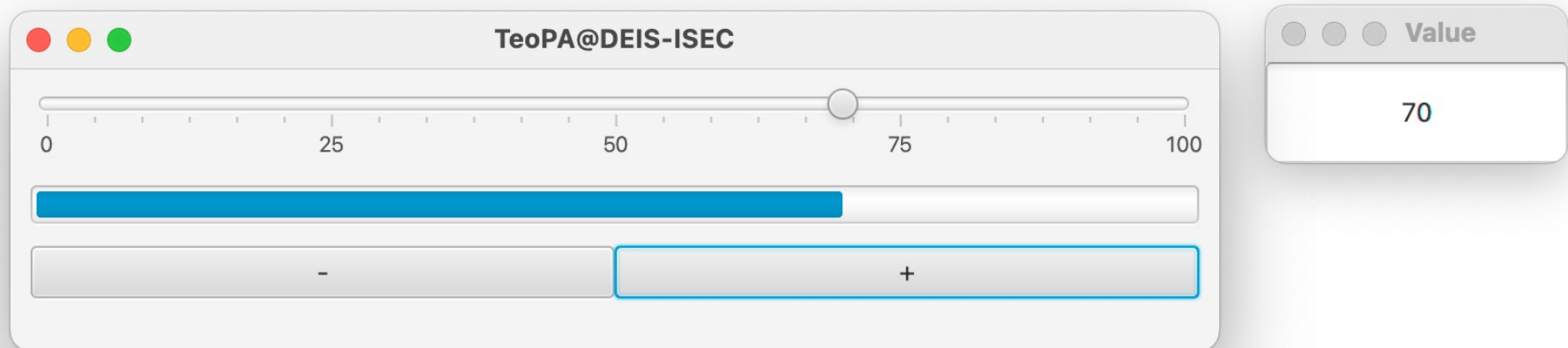
- In the basis presented for the implementation of MVC, it is understood that, every time there is a change to the data, a call to the `update` method must be made so that the view is updated
- The future integration of mechanisms for automatic view updates will simplify this process

MVVM and *Observer/Observable*

- One possible approach is to incorporate certain MVVM features into the project
 - Specifically, we will explore how to integrate the asynchronous capabilities of MVVM into the MVC@PA model, using the *Observer/Observable* pattern and `PropertyChange*` objects
- In the projects developed in this course, the following roles will be used
 - *Observable* – The data model (or an intermediate class, such as a *Facade* or *Proxy*, that acts as an interface to the model)
 - *Observer* – the *View-Controller* classes that need to be notified of changes in the data

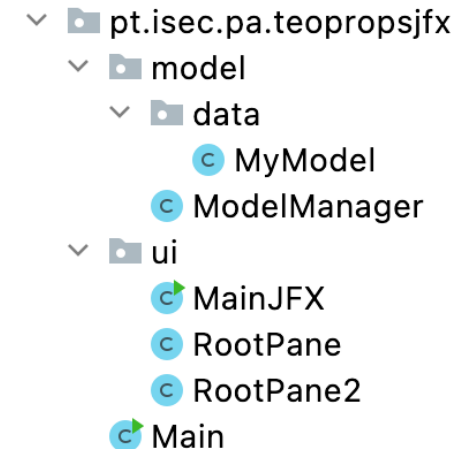
Property Changes example with JavaFX

- Application that automatically updates the interface when a certain model value is changed
 - The value can be changed using a `Slider` component or by direct editing in a separate window
 - Simultaneously it is visualized through a `ProgressBar`



Property Changes example with JavaFX

- Project
 - model
 - data
 - MyModel
 - class to manage the data
 - e.g., an integer that varies between 0 and 100
 - ModelManager
 - class to obfuscate internal model implementation details
 - add the functionality of signaling the property change to the model
 - ui
 - MainJFX
 - *JavaFX* Application class to create the *Stage* objects to support the two windows in the example (one of the *Stage* objects is the one received as a parameter in `start` method)
 - RootPane
 - *Scene graph* object tree of the window with the *Slider* and *ProgressBar*
 - RootPane2
 - *Scene graph* object tree of the window that allows direct editing of the value
 - Main
 - Java class with the `main` method, in which the entire *JavaFX platform* is loaded (`launch`)



MyModel

```
public class MyModel {  
    private int value;  
  
    public MyModel(int value) {  
        this.value = value;  
    }  
  
    public int getValue() {  
        return value;  
    }  
  
    public void setValue(int value) {  
        this.value = Math.min(Math.max(0,value),100);  
    }  
  
    public void inc() {  
        if (value < 100)  
            value++;  
    }  
  
    public void dec() {  
        if (value > 0)  
            value--;  
    }  
}
```

ModelManager

```
public class ModelManager {
    public static final String PROP_VALUE = "prop_value";
    MyModel model;
    PropertyChangeSupport pcs;

    public ModelManager() {
        model = new MyModel(50);
        pcs = new PropertyChangeSupport(this);
    }

    public void addPropertyChangeListener(String property, PropertyChangeListener listener) {
        pcs.addPropertyChangeListener(property, listener);
    }

    public int getValue() { return model.getValue(); }

    public void setValue(int value) {
        int old = model.getValue();
        model.setValue(value);
        pcs.firePropertyChange(PROP_VALUE, old, model.getValue());
    }

    public void inc() {
        model.inc();
        pcs.firePropertyChange(PROP_VALUE, null, null);
    }

    public void dec() {
        model.dec();
        pcs.firePropertyChange(PROP_VALUE, null, null);
    }
}
```

MainJFX e Main

```
public class MainJFX extends Application {
    ModelManager mm;
    public MainJFX() {
        mm = new ModelManager();
    }

    @Override
    public void start(Stage stage) throws Exception {
        RootPane root = new RootPane(mm);
        Scene scene = new Scene(root, 600, 140);
        stage.setScene(scene);
        stage.setTitle("TeoPA@DEIS-ISEC");
        stage.show();

        Stage st2 = new Stage();
        RootPane2 root2 = new RootPane2(mm);
        Scene scene2 = new Scene(root2, 150, 50);
        st2.setScene(scene2);
        st2.setX(stage.getX() + stage.getWidth());
        st2.setY(stage.getY());
        st2.setTitle("Value");
        st2.show();
    }
}
```

```
Main:
public class Main {
    public static void main(String[] args) {
        Application.launch(MainJFX.class, args);
    }
}
```

RootPane

```
public class RootPane extends VBox {
    ModelManager mm;
    Slider slider;
    ProgressBar progressBar;
    Button btnPlus, btnMinus;

    public RootPane(ModelManager mm) {
        this.mm = mm;
        createViews();
        registerHandlers();
        update();
    }
    private void registerHandlers() {
        mm.addPropertyChangeListener(
            ModelManager.PROP_VALUE, evt -> { update(); }
        );

        btnMinus.setOnAction(actionEvent -> { mm.dec(); });
        btnPlus.setOnAction(actionEvent -> { mm.inc(); });
        slider.setOnMouseReleased(mouseEvent -> {
            mm.setValue((int) slider.getValue());
        });
        slider.setOnMouseDragged(mouseEvent -> {
            mm.setValue((int) slider.getValue());
        });
    }
    (...)
}
```

```
(...)
private void createViews() {
    setPadding(new Insets(10));
    setSpacing(10);
    slider = new Slider(0, 100, 50);
    slider.setShowTickMarks(true);
    slider.setShowTickLabels(true);
    slider.setMajorTickUnit(25);
    slider.setMinorTickCount(5);
    slider.setPrefWidth(Integer.MAX_VALUE);
    progressBar = new ProgressBar(0.5);
    progressBar.setPrefWidth(Integer.MAX_VALUE);

    HBox hbox = new HBox();
    btnMinus = new Button("-");
    btnMinus.setPrefWidth(Integer.MAX_VALUE);
    btnPlus = new Button("+");
    btnPlus.setPrefWidth(Integer.MAX_VALUE);
    hbox.setAlignment(Pos.CENTER);
    hbox.getChildren().addAll(btnMinus, btnPlus);
    this.getChildren().addAll(slider, progressBar, hbox);
}

private void update() {
    System.out.println(mm.getValue());
    slider.setValue(mm.getValue());
    progressBar.setProgress(mm.getValue() / 100.0);
}
}
```

RootPane2

```
public class RootPane2 extends TextField {
    ModelManager mm;

    public RootPane2(ModelManager mm) {
        this.mm = mm;
        createViews();
        registerHandlers();
        update();
    }

    private void createViews() { this.setAlignment(Pos.CENTER); }

    private void registerHandlers() {
        mm.addPropertyChangeListener(ModelManager.PROP_VALUE, evt -> { update(); });

        this.setOnKeyPressed(keyEvent -> {
            if (keyEvent.getCode() == KeyCode.ENTER) {
                try {
                    mm.setValue(Integer.parseInt(this.getText()));
                } catch (Exception e) { }
            }
        });
    }

    private void update() { this.setText(""+mm.getValue()); }
}
```

Advanced Programming

JavaFX Resources

Images

Sounds

Fonts

Resources

- Applications may need extra information to...
 - Function properly, or
 - Enrich functionality and user interaction
- This information (usually constituted by auxiliary files) is called *Resources*

Resources

- This information can be of several types, for example:
 - Files with static information to start data structures or databases created by the application
 - Images to improve the representation of information
 - Sounds to provide feedback to user actions
 - Use of fonts best suited to the perception of a given textual information
 - among others...

Resources

- The easiest way to integrate these *Resources* into applications is to provide the files that represent them (the "real resources" themselves) within the application
 - However, the process of providing full paths to the files, to allow them to be opened, is not always simple

Resources

- Through the representative type of a class, obtained using `MyClass.class` or by calling `myObj.getClass()` from a reference to an object of that type, it is possible to have access to methods that facilitate the reading of resources
- For this operation to be feasible, the files must be placed in the same package of these classes or in sub-packages

Resources access

- To get a URL for a resource file
 - `MyClass.class.getResource(String name)`
 - Returns a URL for the resource, for example,
`file:///C:\xpto\otpx.txt`
 - The file path of a file represented by a URL can be obtained using the `getFile()` method
- To get an `InputStream` to the file
 - `MyClass.class.getResourceAsStream(String name)`
 - Returns an `InputStream` that can be used to read the respective information

Example of a utility class

```
public class ResourceLoader {  
    private ResourceLoader() { }  
  
    public static InputStream getResourceFile(String resource) {  
        return ResourceLoader.class.getResourceAsStream(resource);  
    }  
  
    public static String getResourceFilename(String resource) {  
        return ResourceLoader.class.getResource(resource).getFile();  
    }  
}
```

- package pt.isec.pa.xpto
 - package model
 - package ui
 - package resources
 - **class ResourceLoader**
 - package images
 - logo.png
 - diagram.jpg
 - package sounds
 - intro.mp3
 - music.mp3

Example of use:

```
String filename = ResourceLoader.getResourceFilename("images/logo.png");
```

CSS Files

- The styles to apply to *Nodes* belonging to a tree or subtree can be more simply defined using *CSS files*
- Example of a *CSS file* content:

```
.button {  
    -fx-font-family: "Comic Sans MS";  
    -fx-font-size: 16;  
}  
  
#specialButton {  
    -fx-text-fill: red;  
    -fx-font-weight: bold;  
}
```

Helper class for CSS files

```
public class CSSManager {  
    private CSSManager() { }  
  
    public static void applyCSS(Parent parent, String filename) {  
        var url = CSSManager.class.getResource("css/"+filename);  
        if (url == null)  
            return;  
        String fileCSS = url.toExternalForm();  
        parent.getStylesheets().add(fileCSS);  
    }  
}
```

- package pt.isec.pa.xpto
 - package model
 - package ui
 - package resources
 - class CSSManager
 - package css
 - mystyles.css
 - extended.css

Example of use:

```
CSSManager.applyCSS(root, "mystyles.css");
```

Example of using special identifiers (as defined in the previous slide):

```
Button btn = new Button("xpto");  
btn.setId("specialButton");
```

Helper class for fonts

```
public class FontManager {  
    private FontManager() { }  
  
    public static Font loadFont(String filename, int size) {  
        try(InputStream inputStreamFont =  
            FontManager.class.getResourceAsStream("fonts/" + filename)) {  
            return Font.loadFont(inputStreamFont, size);  
        } catch (Exception e) {  
            return null;  
        }  
    }  
}
```

- package pt.isec.pa.xpto
 - package model
 - package ui
 - package resources
 - class FontManager
 - package fonts
 - greatvibes.otf
 - akronim.ttf

Examples of use:

```
Label label = new Label("DEIS");  
label.setFont(FontManager.loadFont("greatvibes.otf", 24));
```

```
GraphicsContext gc = canvas.getGraphicsContext2D();  
gc.setFont(FontManager.loadFont("greatvibes.otf", 48));  
gc.fillText("Advanced Programming", 10, 50);
```


Helper class for images (with cache)

```
public class ImageManager {
    private ImageManager() { }
    private static final HashMap<String, Image> images = new HashMap<>();
    public static Image getImage(String filename) {
        Image image = images.get(filename);
        if (image == null)
            try (InputStream is = ImageManager.class.getResourceAsStream("images/"+filename)) {
                image = new Image(is);
                images.put(filename, image);
            } catch (Exception e) { return null; }
        return image;
    }
    public static Image getExternalImage(String filename) {
        Image image = images.get(filename);
        if (image == null)
            try {
                image = new Image(filename);
                images.put(filename, image);
            } catch (Exception e) { return null; }
        return image;
    }
    public static void purgeImage(String filename) { images.remove(filename); }
}
```

Helper class for images (with cache)

- Example of package organization:

- `package pt.isec.pa.xpto`
 - `package model`
 - `package ui`
 - `package resources`
 - `class ImageManager`
 - `package images`
 - `logo.png`
 - `diagram.jpg`

- Examples of use:

```
GraphicsContext gc = canvas.getGraphicsContext2D();
gc.drawImage(ImageManager.getImage("logo.png"), 150, 150, 200, 100);
gc.drawImage(
    ImageManager.getExternalImage(
        " https://www.oracle.com/img/tech/cb88-java-logo-001.jpg "),
    250, 50, 50, 100);

ImageView imgv = new ImageView(ImageManager.getExternalImage(
    " https://www.oracle.com/img/tech/cb88-java-logo-001.jpg "));
imgv.setPreserveRatio(true);
imgv.setFitWidth(150);
vbox.getChildren().add(imgv);
```

Helper class for sounds

```
public class SoundManager {  
    private SoundManager() { }  
  
    private static MediaPlayer mp;  
  
    public static boolean play(String filename) {  
        try {  
            var url = SoundManager.class.getResource("sounds/" + filename);  
            if (url == null) return false;  
            String path = url.toExternalForm();  
            Media music = new Media(path);  
            if (mp != null && mp.getStatus() == MediaPlayer.Status.PLAYING)  
                mp.stop();  
            mp = new MediaPlayer(music);  
            mp.setStartTime(Duration.ZERO);  
            mp.setStopTime(music.getDuration());  
            mp.setAutoplay(true);  
        } catch (Exception e) {  
            System.err.println("Error playing sound file");  
            return false;  
        }  
        return true;  
    }  
}
```

Helper class for sounds

- Example of package organization:

- `package pt.isec.pa.xpto`
 - `package model`
 - `package ui`
 - `package resources`
 - `class SoundManager`
 - `package sounds`
 - `chicken.mp3`
 - `muttley.mp3`

- Example of use:

```
SoundManager.play("muttley.mp3");
```

Advanced Programming

FSM with JavaFX + Property Change

Example 1: FSM + *JavaFX*

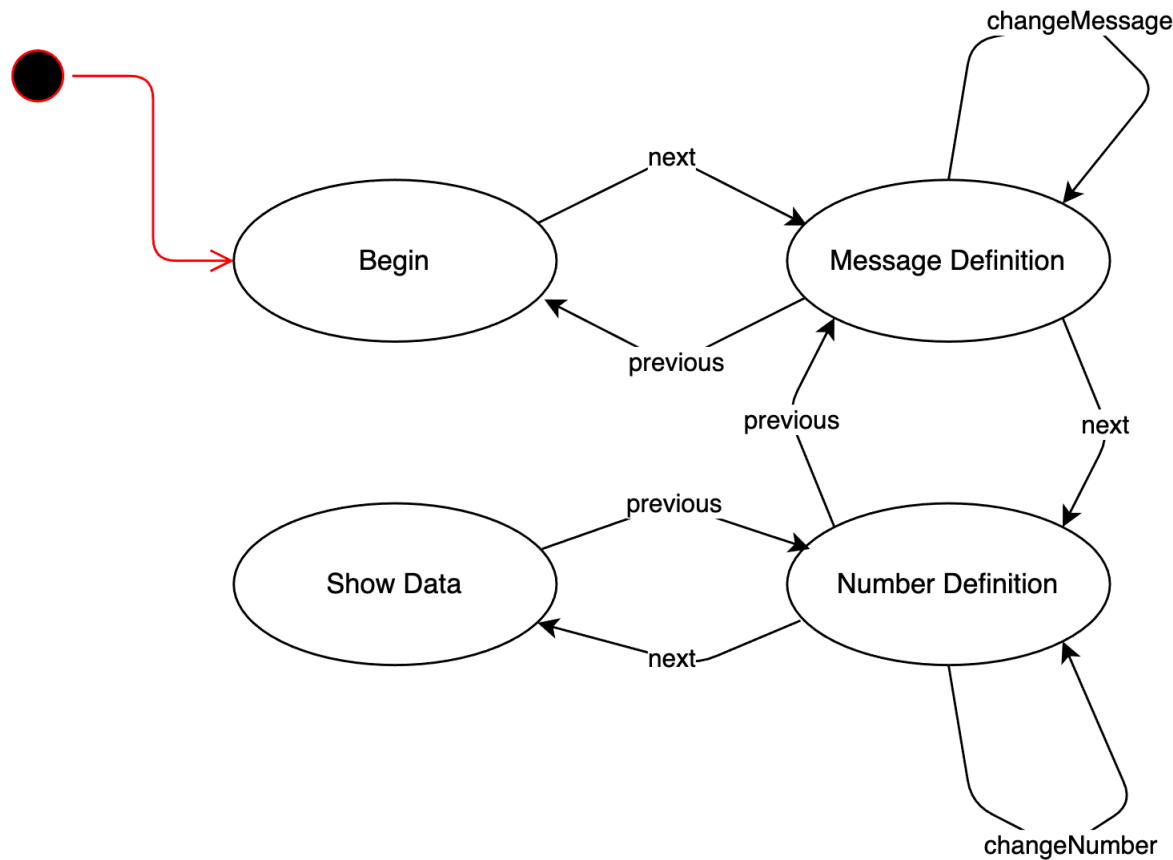
- The following slides will show an application with an academic example, in which:
 - There is a data model that generates an integer value and a *string*
 - *String and integer value* changes must be handled through a state machine, with separate states to change each of the values
 - The current values should always be visible on the screen (in a *status bar*)

FSM with *JavaFX* + *Property Change*

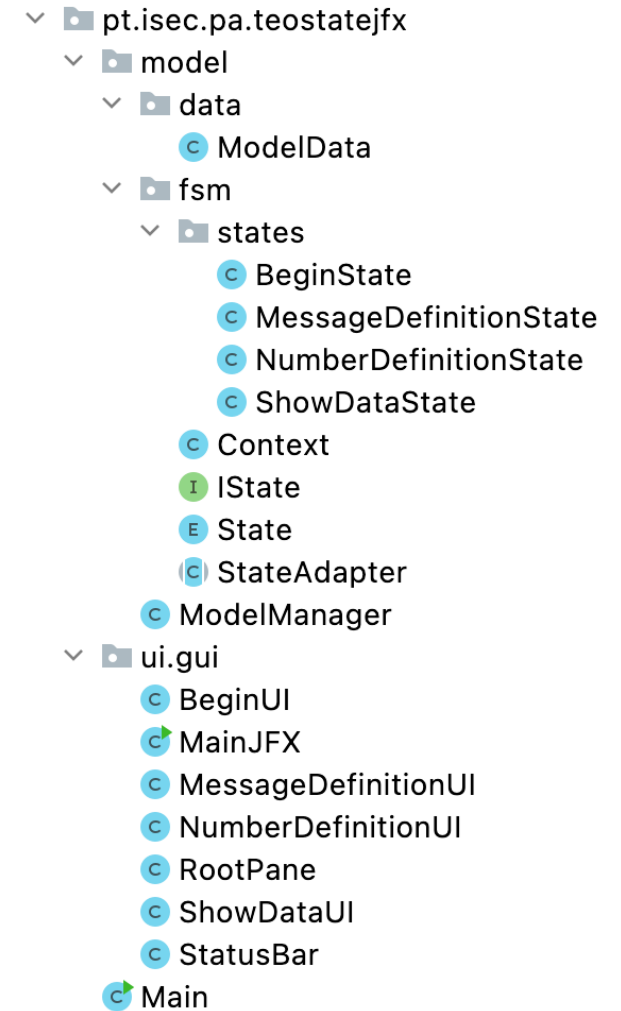
- Creating interfaces for state machines can be accomplished in several ways
- One possibility is...
 - Create a tree of visual elements for each state
 - Similar to the one performed in the previous examples for the `RootPane`, but a new one for each of the states, possibly different from each other
 - Each tree created will alternately occupy the space defined by a `Pane` object or one of its derivatives, placed in the main window of the application for that purpose
 - Using `StackPane` for this purpose could be a good option, since all "*root pane*" will overlap
 - The way to guarantee that only the tree corresponding to the active state is visualized goes through
 - Define a *Property Change* that allows signaling when the active state changes
 - All "*root panes*" in each state manifest interest in this *Property Change*
 - When the event is triggered, all "*root pane*" will be warned of this change
 - If the active state corresponds to the "*root pane*", it will be configured as visible
 - If the active state does not correspond to the "*root pane*", it will be configured as invisible

Example 1: FSM + *JavaFX*

- State diagram



- Project structure



ModelData

```
public class ModelData {  
    String message;  
    int number;  
  
    public ModelData() {  
        message = "none";  
        number = 0;  
    }  
  
    public String getMessage() {  
        return message;  
    }  
  
    public void setMessage(String message) {  
        this.message = message;  
    }  
  
    public int getNumber() {  
        return number;  
    }  
  
    public void setNumber(int number) {  
        this.number = number;  
    }  
}
```

State, IState

```
public enum State {
    BEGIN, MESSAGE_DEFINITION, NUMBER_DEFINITION, SHOW_DATA;

    IState createState(Context context, ModelData data) {
        return switch (this) {
            case BEGIN                -> new BeginState(context,data);
            case MESSAGE_DEFINITION    -> new MessageDefinitionState(context,data);
            case NUMBER_DEFINITION     -> new NumberDefinitionState(context,data);
            case SHOW_DATA             -> new ShowDataState(context,data);
        };
    }
}
```

```
public interface IState {
    void changeMessage(String msg);
    void changeNumber(int nr);
    void next();
    void previous();

    State getState();
}
```

StateAdapter

```
public abstract class StateAdapter implements IState {  
    protected Context context;  
    protected ModelData data;  
  
    protected StateAdapter(Context context, ModelData data) {  
        this.context = context;  
        this.data = data;  
    }  
  
    protected void changeState(State newState) {  
        context.changeState(newState.createState(context, data));  
    }  
  
    @Override  
    public void changeMessage(String msg) { }  
  
    @Override  
    public void changeNumber(int nr) { }  
  
    @Override  
    public void next() { }  
  
    @Override  
    public void previous() { }  
}
```

BeginState

```
public class BeginState extends StateAdapter {  
    public BeginState(Context context, ModelData data) {  
        super(context, data);  
    }  
  
    @Override  
    public void next() {  
        changeState(State.MESSAGE_DEFINITION);  
    }  
  
    @Override  
    public State getState() {  
        return State.BEGIN;  
    }  
}
```

MessageDefinitionState

```
public class MessageDefinitionState extends StateAdapter {  
    public MessageDefinitionState(Context context, ModelData data) {  
        super(context, data);  
    }  
  
    @Override  
    public void next() { changeState(State.NUMBER_DEFINITION); }  
  
    @Override  
    public void previous() { changeState(State.BEGIN); }  
  
    @Override  
    public void changeMessage(String msg) { data.setMessage(msg); }  
  
    @Override  
    public State getState() { return State.MESSAGE_DEFINITION; }  
}
```

NumberDefinitionState

```
public class NumberDefinitionState extends StateAdapter {  
    public NumberDefinitionState(Context context, ModelData data) {  
        super(context, data);  
    }  
  
    @Override  
    public void next() { changeState(State.SHOW_DATA); }  
  
    @Override  
    public void previous() { changeState(State.MESSAGE_DEFINITION); }  
  
    @Override  
    public void changeNumber(int nr) { data.setNumber(nr); }  
  
    @Override  
    public State getState() { return State.NUMBER_DEFINITION; }  
}
```

ShowDataState

```
public class ShowDataState extends StateAdapter {  
    public ShowDataState(Context context, ModelData data) {  
        super(context, data);  
    }  
  
    @Override  
    public void previous() {  
        changeState(State.NUMBER_DEFINITION);  
    }  
  
    @Override  
    public State getState() {  
        return State.SHOW_DATA;  
    }  
}
```

Context

```
public class Context {
    ModelData data;
    IState state;

    public Context() {
        data = new ModelData();
        state = State.BEGIN.createState(data);
    }

    void changeState(IState state)        { this.state = state; }

    public State getState()                { return state.getState(); }

    public void next()                     { state.next(); }

    public void previous()                  { state.previous(); }

    public void changeMessage(String msg) { state.changeMessage(msg); }

    public void changeNumber(int nr)       { state.changeNumber(nr); }

    public int getNumber()                  { return data.getNumber(); }

    public String getMessage()              { return data.getMessage(); }
}
```


ModelManager

```
public class ModelManager {
    public static final String PROP_STATE = "state";          public static final String PROP_DATA  = "data";

    Context context;
    PropertyChangeSupport pcs;

    public ModelManager() {
        this.context = new Context();
        pcs = new PropertyChangeSupport(this);
    }

    public void addPropertyChangeListener(String property, PropertyChangeListener listener) {
        pcs.addPropertyChangeListener(property, listener);
    }

    public State getState() { return context.getState(); }

    public void next() { context.next(); pcs.firePropertyChange(PROP_STATE, null, context.getState()); }

    public void previous() { context.previous(); pcs.firePropertyChange(PROP_STATE, null, context.getState()); }

    public void changeMessage(String msg) {context.changeMessage(msg); pcs.firePropertyChange(PROP_DATA, null, null); }

    public void changeNumber(int nr) { context.changeNumber(nr); pcs.firePropertyChange(PROP_DATA, null, null); }

    public int getNumber() { return context.getNumber(); }

    public String getMessage() { return context.getMessage(); }
}
```

Main, MainJFX

```
public class Main {  
    public static void main(String[] args) {  
        Application.launch(MainJFX.class,args);  
    }  
}  
  
public class MainJFX extends Application {  
    ModelManager model;  
  
    public MainJFX() { model = new ModelManager(); }  
  
    @Override  
    public void start(Stage stage) throws Exception {  
        RootPane root = new RootPane(model);  
        Scene scene = new Scene(root,600,400);  
        stage.setScene(scene);  
        stage.setTitle("TeoStateJFX");  
        stage.setMinWidth(400);  
        stage.show();  
    }  
}
```

RootPane

```
public class RootPane extends BorderPane {
    ModelManager model;

    public RootPane(ModelManager model) {
        this.model = model;

        createViews();
        registerHandlers();
        update();
    }

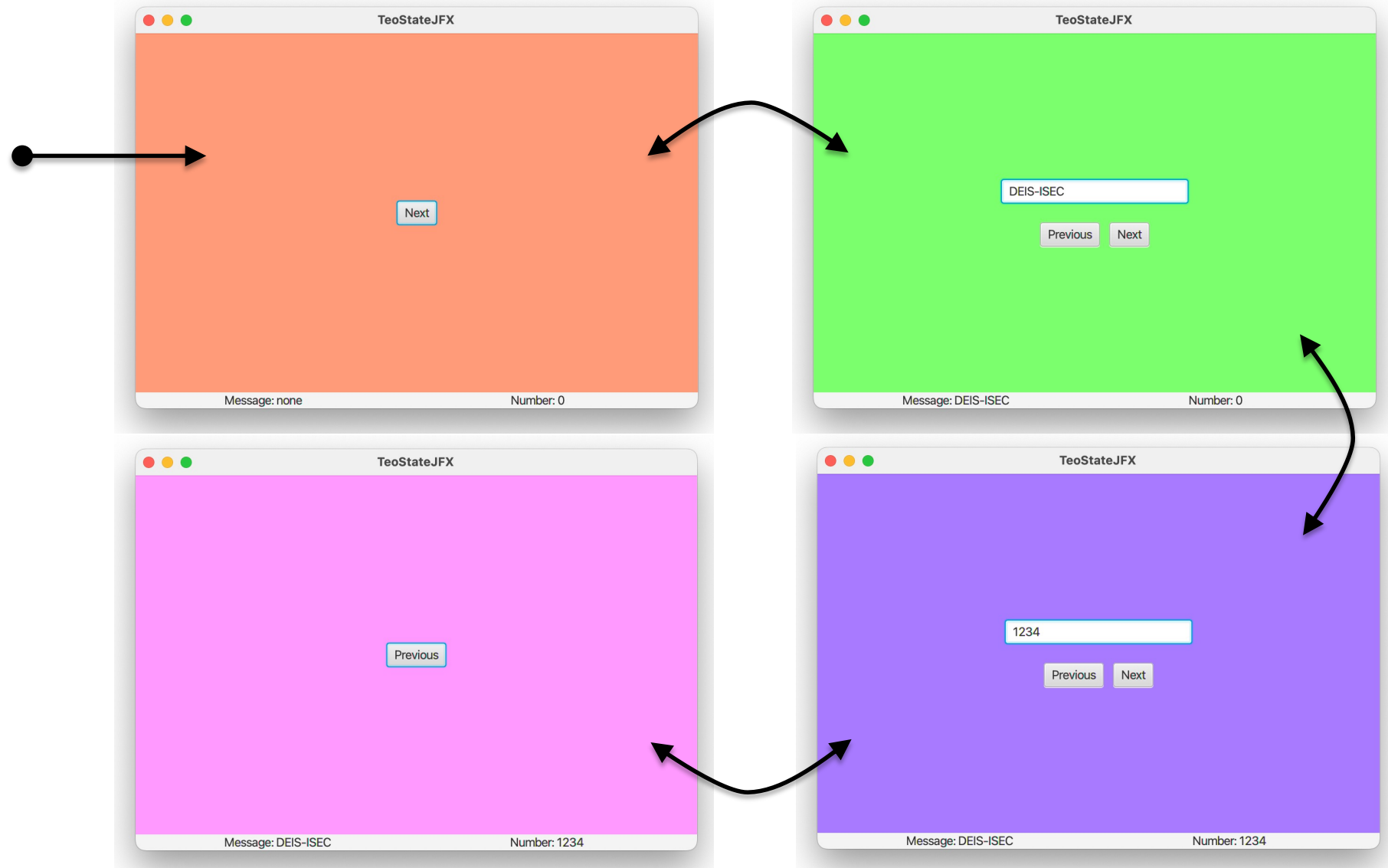
    private void createViews() {
        StackPane stackPane = new StackPane(
            new BeginUI(model),
            new MessageDefinitionUI(model),
            new NumberDefinitionUI(model),
            new ShowDataUI(model)
        );
        this.setCenter(stackPane);

        this.setBottom(new StatusBar(model));
    }

    private void registerHandlers() { }

    private void update() { }
}
```

Application UI



BeginUI

```
public class BeginUI extends BorderPane {
    ModelManager model;
    Button btnNext;

    public BeginUI(ModelManager model) {
        this.model = model;
        createViews();
        registerHandlers();
        update();
    }

    private void createViews() {
        this.setStyle("-fx-background-color: #FFA080;");
        btnNext = new Button("Next");
        this.setCenter(btnNext);
    }

    private void registerHandlers() {
        model.addPropertyChangeListener(ModelManager.PROP_STATE, evt -> { update(); });
        btnNext.setOnAction(actionEvent -> { model.next(); });
    }

    private void update() {
        this.setVisible(model.getState() == State.BEGIN);
    }
}
```

MessageDefinitionUI

```
public class MessageDefinitionUI extends BorderPane {
    ModelManager model;
    Button btnNext, btnPrevious;
    TextField tfMsg;

    public MessageDefinitionUI(ModelManager model) {
        this.model = model;
        createViews();
        registerHandlers();
        update();
    }

    private void createViews() {
        this.setStyle(
            "-fx-background-color: #A0FF80;");
        btnNext = new Button("Next");
        btnPrevious = new Button("Previous");
        HBox hbox = new HBox(btnPrevious, btnNext);
        hbox.setSpacing(10);
        hbox.setAlignment(Pos.CENTER);
        tfMsg = new TextField();
        tfMsg.setMaxWidth(200);
        VBox vbox = new VBox(tfMsg, hbox);
        vbox.setSpacing(20);
        vbox.setAlignment(Pos.CENTER);
        this.setCenter(vbox);
    }
}
```

(...)

```
(...)

private void registerHandlers() {
    model.addPropertyChangeListener(
        ModelManager.PROP_STATE, evt -> {
            update();
        });
    btnNext.setOnAction(actionEvent -> {
        model.next();
    });
    btnPrevious.setOnAction(actionEvent -> {
        model.previous();
    });
    tfMsg.setOnKeyTyped(keyEvent -> {
        model.changeMessage(tfMsg.getText());
    });
}

private void update() {
    this.setVisible(
        model.getState() == State.MESSAGE_DEFINITION
    );
    tfMsg.setText(model.getMessage());
    tfMsg.requestFocus();
}
}
```

NumberDefinitionUI

```
public class NumberDefinitionUI extends BorderPane {
    ModelManager model;
    Button btnNext, btnPrevious;
    TextField tfNumber;

    public NumberDefinitionUI(ModelManager model) {
        this.model = model;
        createViews();
        registerHandlers();
        update();
    }

    private void createViews() {
        this.setStyle(
            "-fx-background-color: #A080FF;");
        btnNext = new Button("Next");
        btnPrevious = new Button("Previous");
        HBox hbox = new HBox(btnPrevious, btnNext);
        hbox.setSpacing(10);
        hbox.setAlignment(Pos.CENTER);
        tfNumber = new TextField();
        tfNumber.setMaxWidth(200);
        VBox vbox = new VBox(tfNumber, hbox);
        vbox.setSpacing(20);
        vbox.setAlignment(Pos.CENTER);
        this.setCenter(vbox);
    }
}
```

(...)

```
(...)

private void registerHandlers() {
    model.addPropertyChangeListener(
        ModelManager.PROP_STATE, evt -> {
            update();
        });
    btnNext.setOnAction(actionEvent -> {
        model.next();
    });
    btnPrevious.setOnAction(actionEvent -> {
        model.previous();
    });
    tfNumber.setOnKeyTyped(keyEvent -> {
        try {
            model.changeNumber(
                Integer.parseInt(tfNumber.getText())
            );
        } catch (Exception e) {
            model.changeNumber(-1);
        }
    });
}

private void update() {
    this.setVisible(
        model.getState() == State.NUMBER_DEFINITION);
    tfNumber.setText("" + model.getNumber());
    tfNumber.requestFocus();
}
}
```

ShowDataUI

```
public class ShowDataUI extends BorderPane {
    ModelManager model;
    Button btnPrevious;

    public ShowDataUI(ModelManager model) {
        this.model = model;
        createViews();
        registerHandlers();
        update();
    }

    private void createViews() {
        this.setStyle("-fx-background-color: #FFA0FF;");
        btnPrevious = new Button("Previous");
        this.setCenter(btnPrevious);
    }

    private void registerHandlers() {
        model.addPropertyChangeListener(ModelManager.PROP_STATE, evt -> { update(); });
        btnPrevious.setOnAction(actionEvent -> { model.previous(); });
    }

    private void update() {
        this.setVisible(model.getState() == State.SHOW_DATA);
    }
}
```


StatusBar

```
public class StatusBar extends HBox {
    ModelManager model;
    Label lbMsg, lbNumber;

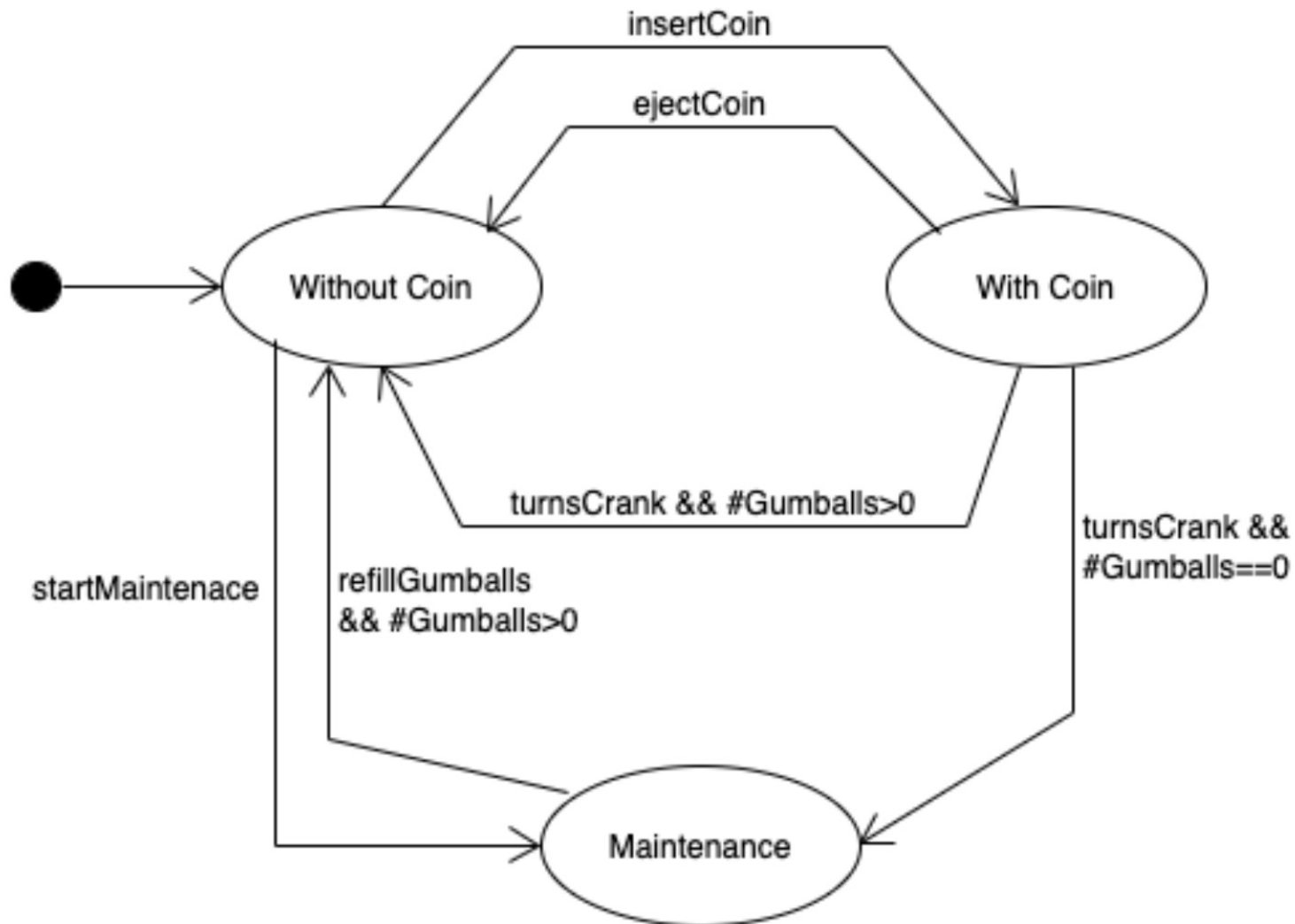
    public StatusBar(ModelManager model) {
        this.model = model;
        createViews();    registerHandlers();    update();
    }
    private void createViews() {
        Label lbMsgTitle = new Label("Message: ");
        lbMsgTitle.setPrefWidth(Integer.MAX_VALUE);
        lbMsgTitle.setAlignment(Pos.CENTER_RIGHT);
        lbMsg = new Label();
        lbMsg.setPrefWidth(Integer.MAX_VALUE);
        lbMsg.setAlignment(Pos.CENTER_LEFT);
        Label lbNumberTitle = new Label("Number: ");
        lbNumberTitle.setPrefWidth(Integer.MAX_VALUE);
        lbNumberTitle.setAlignment(Pos.CENTER_RIGHT);
        lbNumber = new Label();
        lbNumber.setPrefWidth(Integer.MAX_VALUE);
        lbNumber.setAlignment(Pos.CENTER_LEFT);
        this.getChildren().addAll(lbMsgTitle, lbMsg, lbNumberTitle, lbNumber);
    }
    private void registerHandlers() { model.addPropertyChangeListener(ModelManager.PROP_DATA, e->{update();}); }
    private void update() {
        lbMsg.setText(model.getMessage());
        lbNumber.setText("" + model.getNumber());
    }
}
```

Remember – "Gumball Machine"

- Let's assume that a simulator of a gumball machine is to be implemented...
- The gumball machine dispenses gumballs when a coin is inserted and the crank/handle is turned
- Actions (all actions allowed regardless of state):
 - Insert coin
 - Withdraw/eject coin
 - Turn crank
 - Put in maintenance mode/Start maintenance
 - Refill gumballs
- States
 - Without coin
 - With coin
 - In maintenance mode



Remember – State diagram



Remember – GumballMachineData

```
public class GumballMachineData {  
    private int count = 0;  
  
    public GumballMachineData(int count) { this.count = count; }  
  
    public int getCount() { return count; }  
  
    public void refillGumballs(int count) {  
        if (count>0)  
            this.count += count;  
    }  
  
    public boolean getGumball() {  
        if (count>0) {  
            count--;  
            return true;  
        }  
        return false;  
    }  
  
    @Override  
    public String toString() {  
        return String.format("Gumball Machine with %d gumball(s)", count);  
    }  
}
```



Remember – IState and enum State

```
public enum State {  
    MAINTENANCE, WITH_COIN, WITHOUT_COIN  
}
```

```
public interface IState {  
    boolean insertCoin();  
    boolean ejectCoin();  
    boolean turnsCrank();  
    boolean startMaintenance();  
    boolean refillGumballs(int count);  
  
    State getState();  
}
```

Remember – StateAdapter

```
abstract class StateAdapter implements IState {
    protected Context context;
    protected GumballMachineData data;

    protected StateAdapter(Context context, GumballMachineData data) {
        this.context = context;
        this.data = data;
    }

    protected void changeState(IState newState) { context.changeState(newState); }

    @Override
    public boolean insertCoin() { return false; }

    @Override
    public boolean ejectCoin() { return false; }

    @Override
    public boolean turnsCrank() { return false; }

    @Override
    public boolean startMaintenance() { return false; }

    @Override
    public boolean refillGumballs(int count) { return false; }
}
```

Remember – WithoutCoinState

```
class WithoutCoinState extends StateAdapter {
    WithoutCoinState(Context context, GumballMachineData data) {
        super(context,data);
    }

    @Override
    public boolean insertCoin() {
        changeState(new WithCoinState(context,data));
        return true;
    }

    @Override
    public boolean startMaintenance() {
        changeState(new MaintenanceState(context,data));
        return true;
    }

    @Override
    public State getState() { return State.WITHOUT_COIN; }
}
```

Remember – WithCoin

```
class WithCoinState extends StateAdapter {
    WithCoinState(Context context, GumballMachineData data) {
        super(context, data);
    }

    @Override
    public boolean ejectCoin() {
        changeState(new WithoutCoinState(context, data));
        return true;
    }

    @Override
    public boolean turnsCrank() {
        if (data.getGumball() && data.getCount() > 0) {
            changeState(new WithoutCoinState(context, data));
            return true;
        }
        changeState(new MaintenanceState(context, data));
        return false;
    }

    @Override
    public State getState() { return State.WITH_COIN; }
}
```


Remember – MaintenanceState

```
class MaintenanceState extends StateAdapter {
    MaintenanceState(Context context, GumballMachineData data) {
        super(context,data);
    }

    @Override
    public boolean refillGumballs(int count) {
        data.refillGumballs(count);
        if (data.getCount()>0) {
            changeState(new WithoutCoinState(context, data));
            return true;
        }
        return false;
    }

    @Override
    public State getState() {
        return State.MAINTENANCE;
    }
}
```

Remember – *State Machine Context*

```
public class Context {
    private GumballMachineData data;
    private IState state;

    public Context(int count) {
        data = new GumballMachineData(count);
        state = new WithoutCoinState(this, data);
    }

    public State getState() {
        return state.getState();
    }

    void changeState(IState newState) {
        this.state = newState;
    }

    // ---- get data ----

    public int getCount() {
        return data.getCount();
    }

    // ...

    // ...

    // Transitions
    public boolean insertCoin() {
        return state.insertCoin();
    }

    public boolean ejectCoin() {
        return state.ejectCoin();
    }

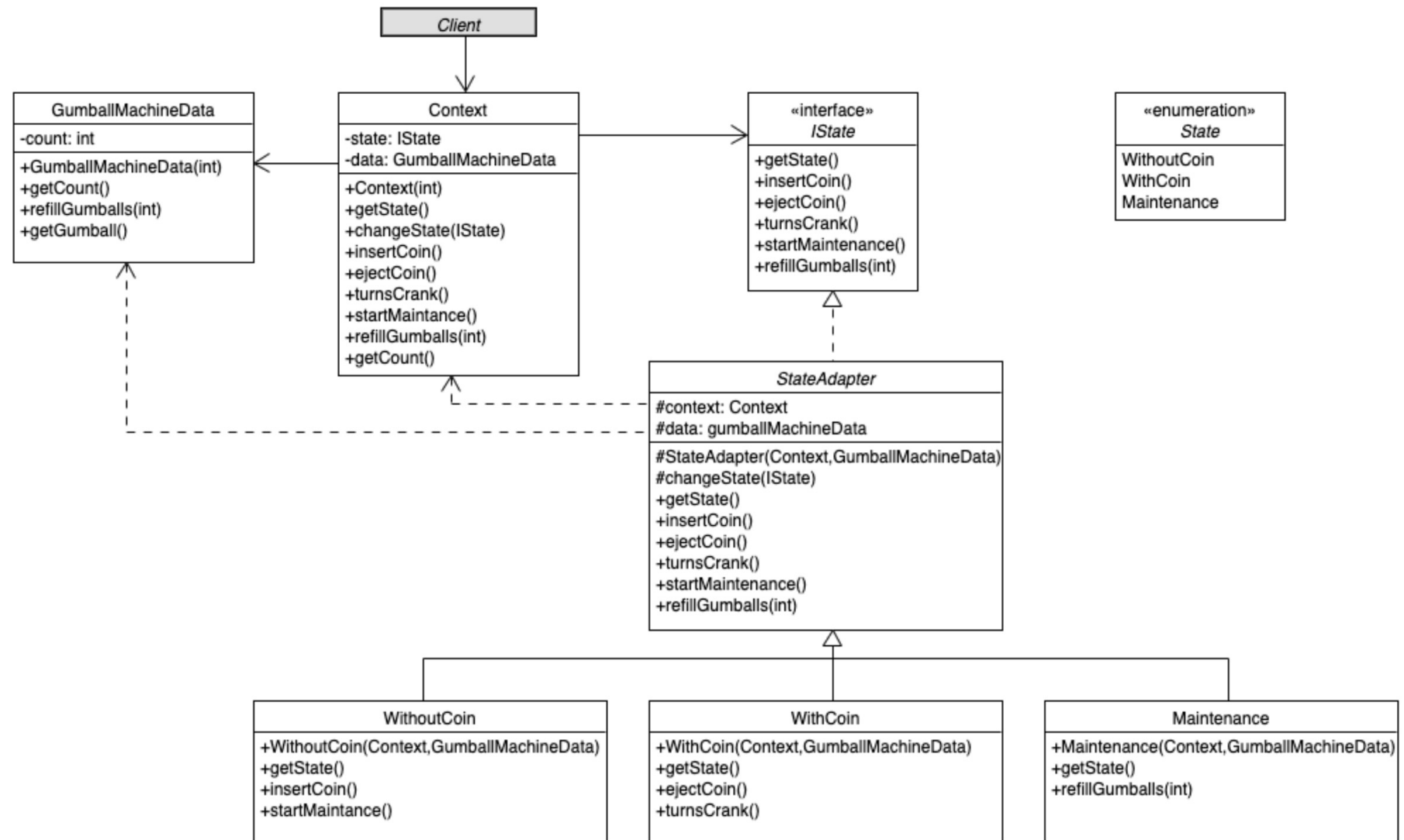
    public boolean turnsCrank() {
        return state.turnsCrank();
    }

    public boolean startMaintenance() {
        return state.startMaintenance();
    }

    public boolean refillGumballs(int count)
    {
        return state.refillGumballs(count);
    }

}
```

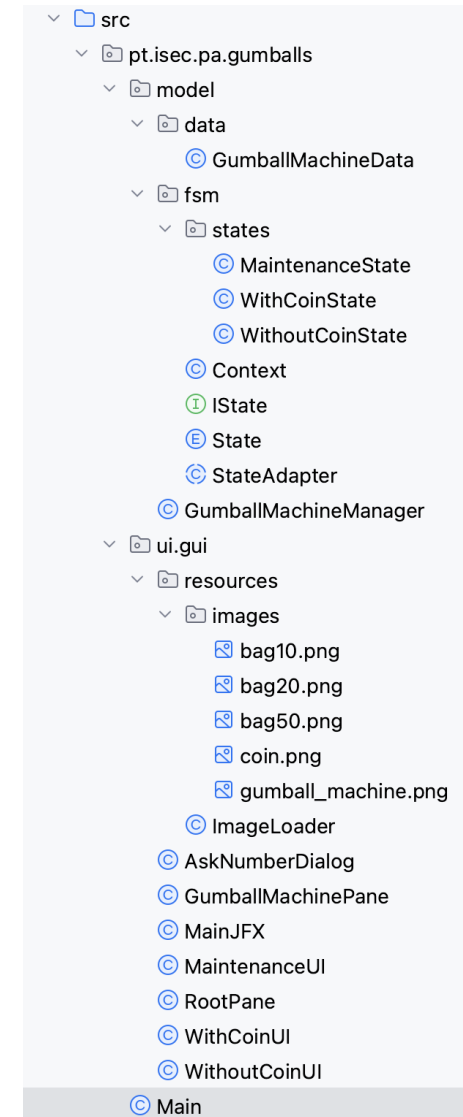
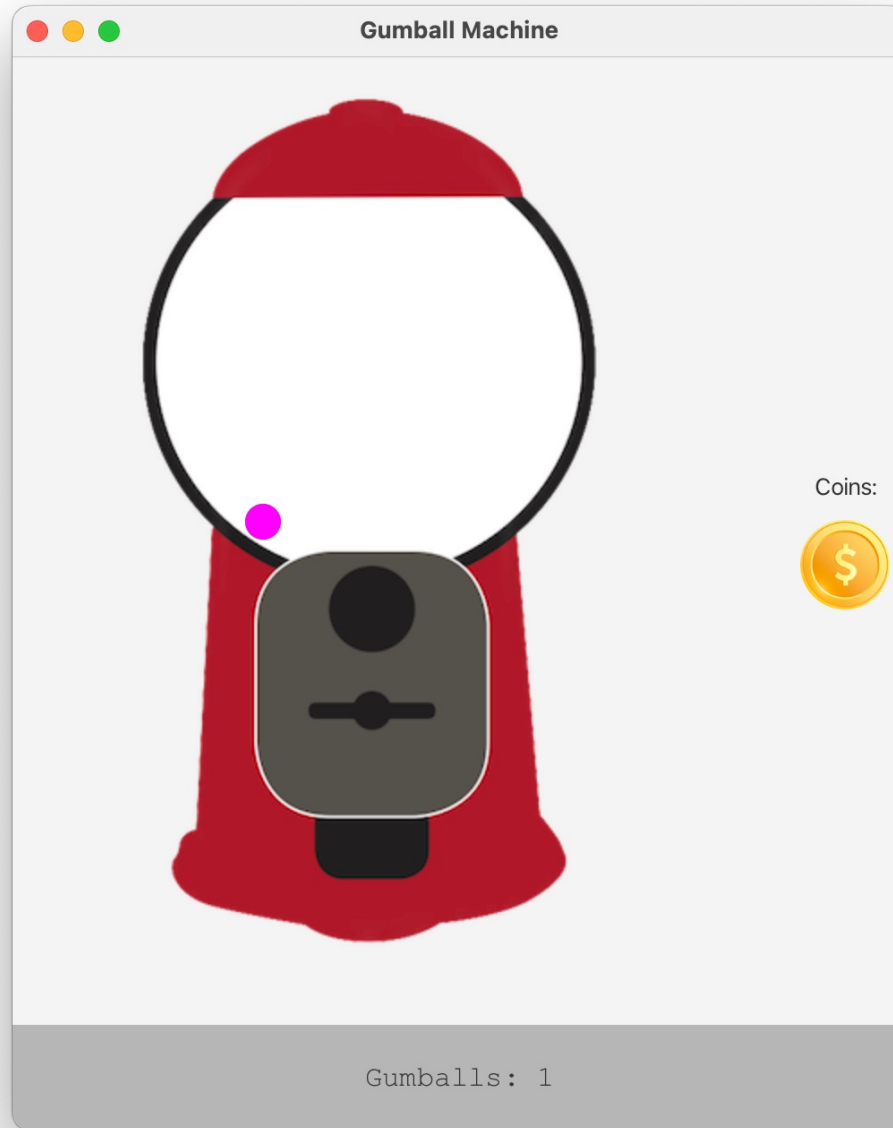
Remember – "Gumball Machine v1.0"



"Gumball Machine v2.0", using JavaFX

```
public class Main {  
  
    public static GumballMachineManager model = null;  
    //eventually, a multiton can be used (based on a stage object)  
  
    public static void main(String[] args) {  
        model = new GumballMachineManager(1); // only one for tests  
        Application.launch(MainJFX.class, args);  
    }  
}
```

"Gumball Machine v2.0", using JavaFX



GumballMachineManager

```
public class GumballMachineManager {
    public static final String PROP_STATE = "_state_";
    public static final String PROP_UIUPDATE = "_ui_";
    Context fsm;
    PropertyChangeSupport pcs;

    public GumballMachineManager(int nrBalls) {
        fsm = new Context(nrBalls);
        pcs = new PropertyChangeSupport(fsm);
    }

    // observer/observable management
    public void addClient(String p, PropertyChangeListener l) {
        pcs.addPropertyChangeListener(p,l);
    }

    public void removeClient(String p, PropertyChangeListener l) {
        pcs.removePropertyChangeListener(p, listener);
    }

    // ---- get data ----
    public State getState() {
        return fsm.getState();
    }

    public int getCount() {
        return fsm.getCount();
    }
... ==>
```

```
... ==>
    // Transitions
    public boolean insertCoin() {
        boolean ret = fsm.insertCoin();
        pcs.firePropertyChange(PROP_STATE,null,null);
        return ret;
    }

    public boolean ejectCoin() {
        boolean ret = fsm.ejectCoin();
        pcs.firePropertyChange(PROP_STATE,null,null);
        return ret;
    }

    public boolean turnsCrank() {
        boolean ret = fsm.turnsCrank();
        pcs.firePropertyChange(PROP_STATE,null,null);
        pcs.firePropertyChange(PROP_UIUPDATE,null,null);
        return ret;
    }

    public boolean startMaintenance() {
        boolean ret = fsm.startMaintenance();
        pcs.firePropertyChange(PROP_STATE,null,null);
        pcs.firePropertyChange(PROP_UIUPDATE,null,null);
        return ret;
    }

    public boolean refillGumballs(int count) {
        boolean ret = fsm.refillGumballs(count);
        pcs.firePropertyChange(PROP_STATE,null,null);
        pcs.firePropertyChange(PROP_UIUPDATE,null,null);
        return ret;
    }
}
```

MainJFX

```
public class MainJFX extends Application {
    GumballMachineManager model;

    public MainJFX() {
        this.model = Main.model;
    }

    @Override
    public void start(Stage stage) throws Exception {
        RootPane root = new RootPane(model);
        Scene scene = new Scene(root, 500, 600);
        stage.setTitle("Gumball Machine");
        stage.setScene(scene);
        stage.setResizable(false);
        stage.show();
    }
}
```

RootPane

```
public class RootPane extends BorderPane {
    GumballMachineManager model;
    StackPane stackPane;
    HBox statusBar;
    Label lblBalls;

    public RootPane(GumballMachineManager model) {
        this.model = model;
        createViews();          registerHandlers();          update();
    }
    private void createViews() {
        stackPane = new StackPane(new WithoutCoinUI(model), new WithCoinUI(model), new MaintenanceUI(model));
        setCenter(stackPane);
        lblBalls = new Label();
        statusBar = new HBox(lblBalls);
        statusBar.setPadding(new Insets(20));
        statusBar.setBackground(new Background(new BackgroundFill(
                                                    Color.GRAY.brighter(), CornerRadii.EMPTY, Insets.EMPTY)));
        statusBar.setAlignment(Pos.BASELINE_CENTER);
        lblBalls.setFont(new Font("Courier new", 16));
        setBottom(statusBar);
    }
    private void registerHandlers() { model.addClient(GumballMachineManager.PROP_UIUPDATE, e-> update()); }

    private void update() { lblBalls.setText("Gumballs: "+model.getCount()); }
}
```

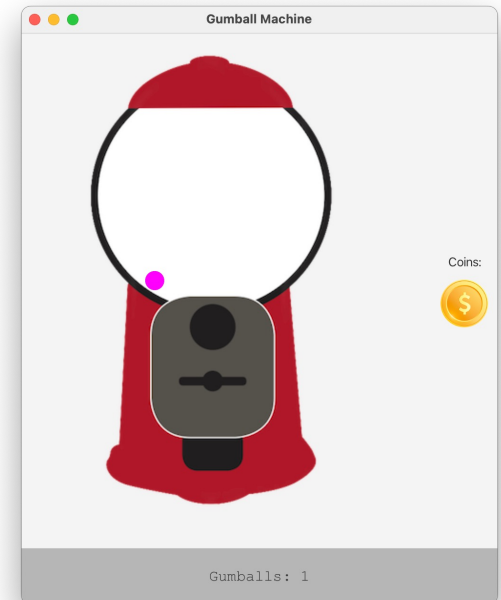

WithoutCoinUI

```
public class WithoutCoinUI extends BorderPane {
    GumballMachineManager model;
    ImageView coinImage;

    public WithoutCoinUI(GumballMachineManager model) {
        this.model = model;
        createViews();          registerHandlers();          update();
    }
    private void createViews() {
        GumballMachinePane gumballMachinePane = new GumballMachinePane(model, null, null);
        setCenter(gumballMachinePane);

        coinImage = new ImageView(ImageLoader.getImage("coin.png"));
        coinImage.setFitHeight(50);          coinImage.setFitWidth(50);

        Label lblCoins = new Label("Coins:");
        lblCoins.setAlignment(Pos.CENTER);
        VBox vBox = new VBox(lblCoins, coinImage);
        vBox.setPrefWidth(70);          vBox.setFillWidth(true);          vBox.setAlignment(Pos.CENTER);
        vBox.setSpacing(10);
        setRight(vBox);
    }
    private void registerHandlers() {
        model.addClient(GumballMachineManager.PROP_STATE, evt -> update());
        coinImage.setOnMouseClicked( m -> model.insertCoin());
    }
    private void update() { this.setVisible(model.getState() == State.WITHOUT_COIN); }
}
```



WithCoinUI

```
public class WithCoinUI extends BorderPane {
    GumballMachineManager model;

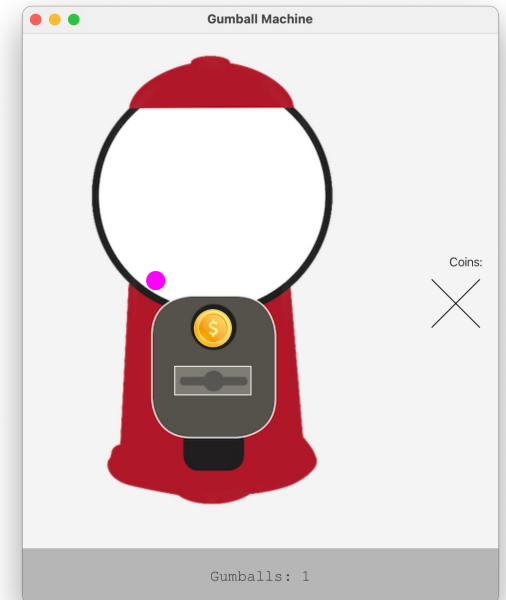
    public WithCoinUI(GumballMachineManager model) {
        this.model = model;
        createViews();          registerHandlers();          update();
    }

    private void createViews() {
        GumballMachinePane gumballMachinePane = new GumballMachinePane(
            model, mouseEvent -> model.ejectCoin(), mouseEvent -> model.turnsCrank()
        );
        setCenter(gumballMachinePane);

        Pane noCoin = new Pane();
        noCoin.setPrefSize(50,50);
        noCoin.getChildren().addAll( new Line(0,0,50,50), new Line(50,0,0,50) );
        Label lblCoins = new Label("Coins:");
        lblCoins.setAlignment(Pos.CENTER);
        VBox vbox = new VBox(lblCoins,noCoin);
        vbox.setPrefWidth(70);          vbox.setFillWidth(true);          vbox.setAlignment(Pos.CENTER);
        vbox.setSpacing(10);
        setRight(vbox);
    }

    private void registerHandlers() {
        model.addClient(GumballMachineManager.PROP_STATE, evt -> update());
    }

    private void update() { this.setVisible(model.getState() == State.WITH_COIN); }
}
```



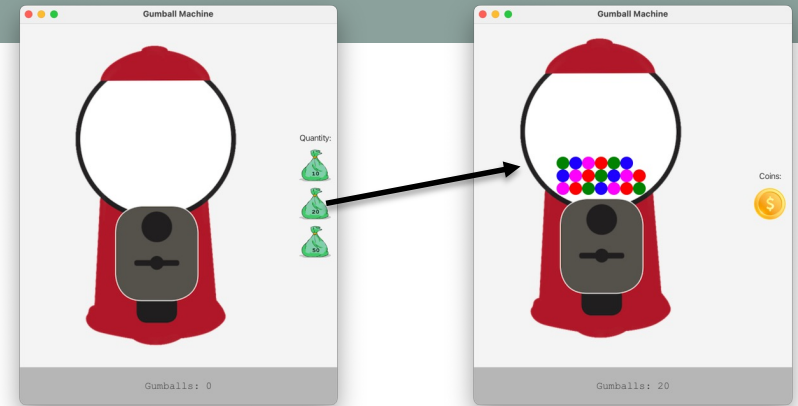
MaintenanceUI

```
public class MaintenanceUI extends BorderPane {
    GumballMachineManager model;
    ImageView bag10, bag20, bag50;

    public MaintenanceUI(GumballMachineManager model) {
        this.model = model;
        createViews();          registerHandlers();          update();
    }
    private void createViews() {
        GumballMachinePane gumballMachinePane = new GumballMachinePane(model, null, null);
        setCenter(gumballMachinePane);

        bag10 = new ImageView(ImageLoader.getImage("bag10.png")); bag10.setFitHeight(50); bag10.setFitWidth(50);
        bag20 = new ImageView(ImageLoader.getImage("bag20.png")); bag20.setFitHeight(50); bag20.setFitWidth(50);
        bag50 = new ImageView(ImageLoader.getImage("bag50.png")); bag50.setFitHeight(50); bag50.setFitWidth(50);
        VBox vBox = new VBox(new Label("Quantity:"), bag10, bag20, bag50);
        vBox.setPrefWidth(70);          vBox.setFillWidth(true);          vBox.setAlignment(Pos.CENTER);
        vBox.setSpacing(10);
        setRight(vBox);
    }
    private void registerHandlers() {
        model.addClient(GumballMachineManager.PROP_STATE, evt -> update());
        bag10.setOnMouseClicked(m -> model.refillGumballs(10));
        bag20.setOnMouseClicked(m -> model.refillGumballs(20));
        bag50.setOnMouseClicked(m -> model.refillGumballs(50));
    }

    private void update() { this.setVisible(model.getState() == State.MAINTENANCE); }
}
```



GumballMachinePane

```
public class GumballMachinePane extends Pane {
    private static final Color[] colors = {Color.MAGENTA, Color.RED, Color.GREEN, Color.BLUE};
    GumballMachineManager model;    ImageView gumballMachineImage;    Group regionCoin, regionCrank;
    EventHandler<MouseEvent> coinEvent, crankEvent;
    public GumballMachinePane(GumballMachineManager model,
                              EventHandler<MouseEvent> coinEvent, EventHandler<MouseEvent> crankEvent) {
        this.model = model;          this.coinEvent = coinEvent;          this.crankEvent = crankEvent;
        createViews();              registerHandlers();              update();
    }
    private void createViews() {
        gumballMachineImage = new ImageView(ImageLoader.getImage("gumball_machine.png"));
        this.getChildren().add(gumballMachineImage);

        if (coinEvent!=null) {
            Image image = ImageLoader.getImage("coin.png");
            Circle circle = new Circle(200, 310, 20, new ImagePattern(image));
            regionCoin = new Group(circle); this.getChildren().add(regionCoin);
        }
        if (crankEvent!=null) {
            Rectangle rectangle = new Rectangle(160, 350, 80, 30);
            rectangle.setStroke(Color.color(1,1,0.95));
            rectangle.setFill(Color.color(1,1,0.95,0.25));
            regionCrank = new Group(rectangle); this.getChildren().add(regionCrank);
        }
    }
    ... ==>
```

GumballMachinePane

```
... ==>
private void registerHandlers() {
    model.addClient(GumballMachineManager.PROP_UIUPDATE,e->update());
    if (coinEvent!=null)
        regionCoin.setOnMouseClicked(coinEvent);
    if (regionCoin!=null)
        regionCrank.setOnMouseClicked(crankEvent);
}

private void update() {
    int y = 260;
    int x = 140;
    this.getChildren().removeIf(node -> node instanceof Circle);
    for (int i = 0; i < model.getCount(); i++) {
        Circle ball = new Circle(x, y, 10);
        ball.setFill(colors[i % colors.length]);
        this.getChildren().add(ball);
        x += 20;
        if (x > 260) {
            x = 140;
            y -= 20;
        }
    }
}
}
```

ImageLoader (*Multiton/cache de imagens*)

```
public class ImageLoader {  
    private static final HashMap<String, Image> images = new HashMap<>();  
  
    public static Image getImage(String name) {  
        Image image = images.get(name);  
        if (image == null) {  
            image = new Image(ImageLoader.class.getResourceAsStream("images/" + name));  
            images.put(name, image);  
        }  
        return image;  
    }  
}
```

